

INSTITUTO FEDERAL DE SANTA CATARINA

CÂMPUS GASPAR

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

GUILHERME DI FANTE

Sistema Distribuído de Reservas
Concorrência Sincronização e Distribuição

Relatório técnico apresentado à disciplina de
Programação Concorrente e Distribuída, ministrada pelo
professor Timoteo Monteiro da Silva.

Gaspar
2026

Sumário

- 1 Introdução
- 2 Objetivo
- 3 Arquitetura do sistema
- 4 Protocolo de comunicação
- 5 Experimento de concorrência
- 6 Sistema distribuído e tolerância a falhas
- 7 Respostas às perguntas do relatório
- 8 Limitações
- 9 Conclusão
- 10 Roteiro para demonstração ao vivo
- Referências

1 Introdução

Este relatório apresenta o desenvolvimento de um sistema distribuído para reserva de assentos. O projeto foi implementado em Java e utiliza comunicação TCP para as operações principais, comunicação UDP para consulta rápida de status, múltiplas threads para atendimento simultâneo e duas réplicas de servidor no modelo primária-backup. Além de executar as reservas, o projeto foi preparado para demonstrar uma condição de corrida, aplicar sincronização e comparar os resultados obtidos.

O sistema trabalha com um conjunto fixo de 50 assentos. Cada usuário pode listar os assentos disponíveis, reservar um assento específico, cancelar uma reserva própria, consultar as reservas e encerrar a conexão. Nos testes realizados, três clientes enviaram dez solicitações cada, totalizando 30 operações concorrentes.

2 Objetivo

O objetivo foi compreender na prática como a concorrência pode produzir resultados inconsistentes quando várias threads alteram o mesmo recurso ao mesmo tempo. Também foi necessário corrigir esse problema por meio de sincronização, distribuir o serviço entre duas réplicas, tratar falhas de cliente e servidor e utilizar TCP e UDP em funções diferentes.

3 Arquitetura do sistema

A arquitetura possui clientes TCP, um cliente de status UDP e duas réplicas do servidor. A réplica A inicia como primária nas portas TCP 5000 e UDP 6000. A réplica B inicia como backup nas portas TCP 5001 e UDP 6001. Cada cliente TCP aceito pelo servidor é atendido por uma thread própria.

Componente	Função	Porta ou tecnologia
Cliente TCP	Envia comandos de reserva, cancelamento e consulta	Socket
Cliente UDP	Realiza consulta rápida de status	DatagramSocket
Réplica A	Servidor primário inicial	TCP 5000 e UDP 6000
Réplica B	Servidor de backup e possível primária promovida	TCP 5001 e UDP 6001
Gerenciador de reservas	Mantém assentos, usuários e contador	Memória compartilhada

Fluxo principal: os clientes enviam operações para a réplica A; a réplica A altera o estado e transmite a mudança para a réplica B; se A ficar indisponível, o cliente tenta B; ao atender solicitações normais, B assume o papel de primária promovida.

4 Protocolo de comunicação

As mensagens são textos em UTF-8 e cada mensagem TCP ocupa uma linha. Internamente, os campos são separados pelo caractere |. Para facilitar o uso, o cliente aceita entradas intuitivas, como reservar 5, e converte esse texto para o formato interno RESERVAR|5|usuario.

Comando interno	Função	Exemplo de resposta
LISTAR	Lista assentos disponíveis	LISTA 1,2,3,4,5
RESERVAR assento usuario	Reserva um assento	OK assento reservado
CANCELAR assento usuario	Cancela uma reserva própria	OK reserva cancelada
RESERVADOS	Lista reservas e usuários	RESERVADOS 5:guilherme
STATUS	Consulta estado por TCP	STATUS REPLICA=A PAPEL=PRIMARIA
SAIR	Encerra a conexão	BYE

Entre as réplicas são usados os comandos REPLICAR_RESERVA|assento|usuario e REPLICAR_CANCELAMENTO|assento|usuario. A consulta UDP envia apenas STATUS e recebe informações sobre a réplica, seu papel, a sincronização, a quantidade de reservas e a condição de funcionamento.

5 Experimento de concorrência

O mesmo teste foi executado em duas configurações. Em cada execução foram usados três clientes simultâneos, cada um enviando dez solicitações. Assim, o total esperado era de 30 operações. A diferença entre os testes foi somente a ativação ou desativação da proteção da região crítica.

Experimento	Sincronização	Requisições	Respostas OK	Contador final	Tempo
1	Desativada	30	30	10	167 ms
2	Ativa	30	30	30	358 ms

5.1 Teste sem sincronização

No teste sem sincronização, todas as 30 solicitações receberam resposta de sucesso e os 30 assentos ficaram ocupados, mas o contador compartilhado terminou em 10. Isso aconteceu porque diferentes threads leram o mesmo valor antes de gravar o próximo valor. Como algumas gravações substituíram atualizações feitas por outras threads, ocorreram perdas de atualização.

5.2 Teste com sincronização

No teste com sincronização, o contador terminou em 30, exatamente como esperado. A proteção da região crítica impediu que várias threads executassem ao mesmo tempo o trecho de leitura, alteração e gravação do contador e das reservas. O tempo aumentou de 167 ms para 358 ms, o que é esperado, pois parte das operações passou a aguardar acesso exclusivo ao recurso compartilhado.

6 Sistema distribuído e tolerância a falhas

Foi escolhida a estratégia primária-backup porque ela é simples de implementar e adequada ao tamanho deste projeto. A réplica primária recebe as solicitações dos clientes e envia cada reserva ou cancelamento para o backup. O backup mantém uma cópia do estado e pode continuar o atendimento se a primária falhar.

O teste de falha confirmou esse comportamento. O assento 40 foi reservado enquanto a réplica A era a primária e a alteração foi replicada para B. Depois que A foi encerrada, o cliente se conectou automaticamente a B. A réplica B foi promovida, aceitou a reserva do assento 41 e preservou as reservas anteriores. Também foi testada a desconexão abrupta de um cliente após a reserva do assento 42. O servidor continuou funcionando e outro cliente conseguiu reservar o assento 43.

7 Respostas às perguntas do relatório

7.1 O que caracteriza este sistema como distribuído

O sistema é distribuído porque seus componentes executam em processos separados e se comunicam pela rede. Existem clientes, duas réplicas de servidor e comunicação por sockets TCP e UDP. As réplicas mantêm cópias do mesmo estado e o serviço pode continuar em outra réplica quando a principal falha.

7.2 Recurso compartilhado e região crítica

O recurso compartilhado é o estado das reservas, formado pelos assentos, pelos usuários responsáveis e pelo contador de operações. A região crítica é o trecho que verifica se um assento está livre, registra ou remove a reserva e atualiza o contador. Essas ações precisam funcionar como uma única operação para evitar que duas threads trabalhem sobre o mesmo estado ao mesmo tempo.

7.3 Condição de corrida e produção do problema

Uma condição de corrida acontece quando o resultado depende da ordem e do momento em que as threads acessam um recurso compartilhado. Ela foi produzida desativando a sincronização e executando três clientes ao mesmo tempo, cada um com dez solicitações. As threads conseguiram ler valores iguais do contador antes que as outras terminassem a atualização.

7.4 Inconsistência numérica observada

Foram enviadas 30 solicitações e todas receberam resposta OK, porém o contador terminou em 10 no experimento sem sincronização. Portanto, 20 atualizações do contador foram perdidas devido à concorrência. Os 30 assentos foram ocupados, o que mostrou que a inconsistência estava no valor compartilhado atualizado sem proteção.

7.5 Funcionamento do mecanismo de sincronização

O mecanismo `synchronized` garante exclusão mútua na região crítica. Quando uma thread entra nesse trecho, as outras precisam aguardar. Assim, cada atualização termina antes de a próxima começar, impedindo que uma thread sobrescreva o resultado de outra. Com a sincronização ativa, o contador chegou corretamente a 30.

7.6 Estratégia de replicação utilizada

Foi utilizada a estratégia primária-backup. A réplica A recebe inicialmente as operações e envia as alterações para a réplica B. A escolha foi feita por ser clara, suficiente para duas réplicas e compatível com o objetivo da atividade. Ela também reduz conflitos, pois apenas a primária coordena as alterações durante o funcionamento normal.

7.7 Recuperação após queda de uma réplica

Quando a réplica primária cai, a tentativa de conexão falha e o cliente tenta automaticamente a porta da réplica de backup. Ao receber a solicitação de cliente, o backup é promovido e passa a

responder como servidor principal. Como ele já recebeu as alterações anteriores, o estado das reservas é preservado e não ocorre duplicação dos assentos já reservados.

7.8 Diferença entre TCP e UDP

O TCP é orientado à conexão, garante entrega, preserva a ordem dos dados e informa falhas de comunicação. Por isso, foi utilizado nas operações principais de listar, reservar e cancelar, nas quais a confiabilidade é necessária. O UDP não cria conexão e não garante entrega ou ordem. Ele foi usado na consulta rápida de status, porque uma perda ocasional pode ser resolvida com uma nova consulta sem alterar o estado do sistema.

7.9 Limitações conhecidas

A solução mantém o estado apenas em memória, então a queda simultânea das duas réplicas causa perda das reservas. O failover é simplificado e não utiliza eleição distribuída, quorum ou consenso completo. A réplica de backup é promovida quando recebe uma solicitação normal, mas não existe monitoramento contínuo da primária. Também não foram implementados autenticação, criptografia, persistência em banco de dados, retransmissão automática de pacotes UDP ou recuperação de mudanças realizadas durante uma falha de comunicação entre as réplicas.

8 Limitações

O projeto atende ao objetivo didático, mas não deve ser considerado uma solução pronta para produção. Em um sistema real seriam necessários armazenamento persistente, autenticação dos usuários, comunicação criptografada, confirmação mais robusta da replicação, detecção automática de falhas e prevenção de duas primárias simultâneas. Também seria importante criar testes automatizados adicionais e permitir a recuperação de uma réplica que ficou desatualizada.

9 Conclusão

Os testes mostraram de forma prática que um programa pode responder com sucesso e ainda manter um dado interno incorreto quando várias threads acessam o mesmo recurso sem proteção. No primeiro experimento, 30 operações foram aceitas, mas o contador terminou em 10. Com synchronized, o mesmo teste terminou corretamente em 30. A replicação primária-backup permitiu manter o estado em duas instâncias e continuar o atendimento após a queda da réplica A. A utilização de TCP nas operações principais e UDP na consulta de status também demonstrou que cada protocolo pode ser escolhido conforme a necessidade de confiabilidade da função.

10 Roteiro para demonstração ao vivo

Etapa	Ação	Resultado esperado
1	Iniciar a réplica B e depois a réplica A	B como backup e A como primária
2	Conectar três clientes	Três atendimentos simultâneos em threads
3	Executar 30 solicitações sem sincronização	Contador inconsistente e logs da corrida
4	Repetir com sincronização	Resultado consistente de 30 de 30
5	Consultar STATUS por UDP	Resposta rápida com estado da réplica
6	Encerrar um cliente abruptamente	Servidor continua atendendo
7	Encerrar a réplica A	Cliente reconecta a B e mantém reservas

Referências

ORACLE. Java Platform Standard Edition Documentation. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 28 set. 2026.

INSTITUTO FEDERAL DE SANTA CATARINA. Atividade prática: Sistema de Reservas - Concorrência, Sincronização e Distribuição. Gaspar, 2026.

Repositório do projeto: <https://github.com/GDIFANTE/reservas-distribuidas>